

T2.2 · Primera acción HIGH real · enviar mensaje WhatsApp

Owner: Celestino · 2026-05-11 · branch *pr/t2.2-whapi-high-send-action* · commit local · sin push · sin PR · sin deploy · sin tocar Render.

Resumen ejecutivo: el Action Center pasa a tener su primer ejecutor HIGH real. *preparar_enviar_mensaje_whatsapp* crea una acción en *needs_approval* guardando *numero_destino* + *mensaje* en payload · NUNCA envía al crearse. Solo tras aprobación humana explícita *ejecutar_accion* invoca al proveedor de WhatsApp existente. Idempotencia anti doble envío via *result_json.estado_envio='sent'* persistido inline antes de retornar. Cobro vía T2.1.D · reserva confirma si éxito, libera/reembolsa si falla. Whapi MOCKEADO en tests · ninguna llamada real. **18 tests nuevos · 237 automation total · 0 regresión (la única flake test_listar_jobs_usuario es pre-existente, depende de asyncio.sleep y pasa en aislamiento).**

1. Diseño · preparar → aprobar → confirmar

1.1 Reuso del modelo existente

El tipo de acción *enviar_mensaje_whatsapp* ya existía en *permissions.py* como HIGH y en *costos.py* como 6 créditos. T2.1.A lo dejó intencionalmente SIN ejecutor: *EJECUTORES_T21A* no tenía entrada y *ejecutar_accion* lo rechazaba liberando la reserva. T2.2 agrega el ejecutor faltante sin cambiar riesgo ni costo.

1.2 API pública del nuevo módulo

Función	Rol	Notas
<code>preparar_enviar_mensaje_whatsapp(telefono, numero_destino, mensaje)</code>	PREPARAR · mensaje acción	Valida payload, llama <code><i>crear_accion</i></code> · acción queda en <code><i>needs_approval</i></code>
<code>aprobar_accion(accion_id) · existente</code>	APROBAR · humano	Mismo paso que MEDIUM · transición <code><i>needs_approval → aprobado</i></code>
<code>ejecutar_accion(accion) · existente</code>	CONFIRMAR · disparar envío	llama el ejecutor mapeado · T2.2 mapea <code><i>enviar_mensaje_whatsapp</i></code>
<code>confirmar_enviar_mensaje_whatsapp(accion_id)</code>	Atajo end-to-end	Aprueba (si <code>needs_approval</code>) y ejecuta. Idempotente si la acción ya fue ejecutada

1.3 Flujo del ejecutor real

```
ejecutor_enviar_mensaje_whatsapp(accion, perfil):
    1. parsear payload_json → (numero_destino, mensaje)
       · valida obligatorios · si inválidos → raise ValueError
    2. _leer_result_actual(accion_id) ← lee DB fresca
       · si estado_envio=='sent' AND mensaje_id existe →
         return {**prev, idempotent: True} ← NO invoca provider
    3. _obtener_proveedor() ← override-able en tests
    4. await proveedor.enviar_mensaje(numero_destino, mensaje)
       · si False o lanza → raise RuntimeError
       (ejecutar_accion atraparé y liberará reserva)
    5. resultado = { estado_envio: 'sent', mensaje_id,
                    destino_short, longitud_mensaje, provider, ... }
    6. _persistir_result_inline(accion_id, resultado)
       ← UPDATE result_json antes de retornar ← dedupe en retry
    7. return resultado
```

El paso 6 es el invariante crítico de T2.2: si un crash ocurre DESPUÉS del envío pero ANTES de *marcar_completada*, cualquier retry verá *result_json.estado_envio='sent'* en el paso 2 y NO re-enviará.

2. Archivos creados / modificados

Archivo	Tipo	Resumen
agent/automation/executors/__init__.py	NUEVO	Paquete nuevo · agrupa ejecutores HIGH.
agent/automation/executors/send_message.py	NUEVO ~250 ln	<i>preparar_enviar_mensaje_whatsapp</i>, <i>confirmar_enviar_mensaje</i>
agent/automation/execution.py	MOD ~10 ln	Import del nuevo ejecutor + entrada en <i>EJECUTORES_T21A</i> para <
tests/test_automation_send_message_high.py	NUEVO 18 tests	Preparar · ejecutar sin aprobación · con aprobación · idempotencia en retry
tests/test_automation_execution.py	MOD 1 test	<i>TestEjecutarHighSinEjecutorReal</i> ahora usa <i>contactar_lead</i> (
tests/test_automation_credits_reservas.py	MOD 1 test	<i>test_high_sin_ejecutor_libera_reserva</i> idem.
tests/test_automation_executors_real.py	MOD 1 test	<i>test_high_aprobado_sigue_sin_ejecutor</i> idem.

Sin nuevas migraciones de schema. Se reutilizan campos existentes: *payload_json*, *result_json*, *estado*, *idempotency_key*. Sin nuevos endpoints admin. Sin tocar *permissions.py* ni *costos.py*.

3. Variables de entorno

Cero variables de entorno nuevas. El módulo usa *agent.providers.obtener_proveedor()* que ya lee *WHATSAPP_PROVIDER* y *WHAPI_TOKEN* · NO se tocaron. Tests monkeypatchean *_obtener_proveedor* dentro del módulo *send_message* · ningún test toca *os.environ* de Whapi ni Render.

4. Riesgos y mitigaciones

Riesgo	Sev	Mitigación
Doble envío en retry de worker	Bajo	<code><i>result_json.estado_envio='sent'</i></code> persistido inline antes de retornar. Cualquier 2
Envío sin aprobación humana (auto)	Bajo	<code><i>enviar_mensaje_whatsapp</i></code> es HIGH · <code><i>estado_inicial_para_riesgo</i></code> retorn
Provider falla con saldo descontado	Bajo	T2.1.D ya cubre · <code><i>ejecutar_accion</i></code> atrapa la excepción del ejecutor y llama <code><i></code>
Crash entre <code><i>enviar_mensaje</i></code> OK y <code><i>marcar_completado</i></code> Mitigado parcialmente: <code><i>_persistir_result_inline</i></code> graba <code><i>result_json</i></code> antes		
Adversario inyecta números falsos por payload	Bajo	<code><i>preparar_enviar_mensaje_whatsapp</i></code> valida formato: longitud 6..32 chars · sol
PII filtrada en logs / audit	Bajo	Logs usan <code><i>_short_num</i></code> (prefijo+sufijo). Audit (<code><i>action_completed</i></code>) no rec
Activación accidental en producción	Muy bajo	T2.2 NO añade triggers ni endpoints externos · sólo conecta el ejecutor a un tipo de
CRITICAL confundido con HIGH	Muy bajo	<code><i>envio_masivo_clientes</i></code> sigue siendo CRITICAL · <code><i>esta_bloqueado_t21</i></code> r

5. Tests ejecutados

Suite	Resultado
<code><i>pytest tests/test_automation_send_message_high.py</i></code>	18 passed in 13.51s
<code><i>pytest tests/test_automation_*.py</i></code>	237 passed in 133.57s · +18 vs main
<code><i>pytest</i></code> (suite completa, primera corrida)	899 passed · 0 failed
<code><i>pytest</i></code> (suite completa, segunda corrida)	1 failed (test_jobs · pre-existente · pasa en aislamiento)
<code><i>pytest tests/test_jobs.py::TestEncolarInproc::test_listar_jobs_usuario</i></code> en aislamiento	18 passed in 1.15s

Nota sobre la flake: `test_listar_jobs_usuario` depende de `asyncio.sleep(0.1)` y bajo la carga de la suite completa a veces falla. **No es introducida por T2.2** · existe en main desde antes y no toca nada del Automation Core. La suite específica de T2.2 (18 tests) y la suite de automation (237 tests) son completamente deterministas.

6. Cómo validar localmente

```
git checkout pr/t2.2-whapi-high-send-action

# Tests específicos · todos mockean Whapi · no toca red
pytest tests/test_automation_send_message_high.py -v

# Suite completa de automation
pytest tests/test_automation_*.py

# Smoke en REPL (opcional · usa FakeProveedor para no enviar)
python -c "\
import asyncio, agent.memory as m, agent.automation as a;\
from agent.automation.executors.send_message import \
    preparar_enviar_mensaje_whatsapp, confirmar_enviar_mensaje_whatsapp\
"
```

6.1 Cómo NO validar

No envíes mensajes reales para validar T2.2 todavía. El ejecutor está cableado pero la decisión de exponerlo al LLM (via brain tool) o a admin endpoints externos es un paso separado. Sin un trigger explícito, el ejecutor no se invoca en producción, aunque la branch esté mergeada.

7. Estado git

Item	Valor
Branch local	<i>pr/t2.2-whapi-high-send-action</i>
Base	<i>main</i> · sincronizada antes del branch (commit <i>4633289</i>)
Commit en branch	Por crear · <i>feat(automation): primer ejecutor HIGH real · enviar mensaje WhatsApp (T2.2)</i>
Push	NO realizado · per instrucción
PR	NO abierto · per instrucción
Merge	NO realizado
Deploy	NO realizado
Variables de entorno Render	NO tocadas
AUTOMATION_SCHEDULER_ENABLED	NO activado (sigue ausente · OFF)

8. Siguiente paso recomendado

Antes de mergear T2.2 hay que decidir cómo se va a invocar *preparar_enviar_mensaje_whatsapp* en producción. Cuatro caminos posibles:

Opción	Mecanismo	Cuándo
A	Brain tool · agregar herramienta <i>preparar_enviar_mensaje_whatsapp</i>	Si se quiere un POC de traducción automática en entorno de producción .
B	Admin endpoint · <i>POST /admin/automation/preparar-enviar-mensaje-whatsapp</i>	Si se quiere un POC de traducción automática en entorno de producción .
C	Dashboard del Action Center · botón "Enviar mensaje" en la barra de navegación	Si se quiere un POC de traducción automática en entorno de producción .
D	Solo merge sin trigger · dejar el ejecutor disponible para llamadas reales	Si se quiere un POC de traducción automática en entorno de producción .

Recomendación del agente: opción **D + smoke test pendiente de T2.1.D**. T2.2 mergea con superficie cero (solo conecta el plumbing) · 0 riesgo de envío accidental. Después, en un PR separado (T2.2.A o T2.3), elegir A/B/C según prioridad de producto. Esto permite hacer el smoke pendiente de T2.1.D y validar las reservas + reconciliación en prod sin agregar otra superficie de cambio simultánea.

Recordatorios de seguridad: branch local lista · sin push · sin PR · sin deploy · sin tocar Render · *AUTOMATION_SCHEDULER_ENABLED* sigue ausente (OFF) · ninguna llamada real a Whapi · ningún mensaje enviado.